

MIDTERM PROJECT

Real-Time Data Collection and Stream Processing System

Smart Campus Air Quality Monitoring with Python, Pandas and NumPy

Student: Aliya Kuanysh Group: BDA-2404, Big Data Analytics
Astana IT University

https://github.com/kua14108-code/campus_stream_project?utm_source=chatgpt.com

1. Introduction

The goal of this project is to build a small Python system that works with data while it arrives, not after a full file is ready. The system reads messages from sensors, cleans them, calculates statistics on a moving window and raises an alert when something looks wrong. I used Pandas and NumPy for the analysis, and the code is split into modules so that every step can be tested alone.

I chose a smart campus scenario: air quality monitoring in six rooms of a university building. CO2 is a good example for streaming analysis. It changes slowly when people are in a room, so trends and moving averages are useful. But it can also go wrong quickly, for example when the ventilation stops, so the system must react to events as well.

2. Scenario and Data

2.1 Source, schema and frequency

There is no real sensor network available, so I wrote a data generator (`generator.py`). Each of the six rooms has one sensor that sends a message every 10 seconds. A gateway collects the messages in polling cycles (called polls). The stream has 720 polls, which means 2 hours of simulated time (09:00 to 11:00). Table 1 shows the format of one raw message.

Table 1. Raw message format

Column	Type	Meaning
<code>poll_id</code>	integer	Number of the gateway polling cycle in which the message arrived
<code>timestamp</code>	datetime	Time of the measurement
<code>sensor_id, room</code>	text	Sensor S01 to S06 and the room where it is installed
<code>temperature_c</code>	float	Air temperature, degrees Celsius
<code>humidity_pct</code>	float	Relative humidity, %
<code>co2_ppm</code>	float	CO2 concentration, parts per million

The CO2 value follows a simple physical model. It grows when people are in the room and falls because of ventilation: $dCO_2/dt = \text{generation}(\text{people}) - \text{ventilation} \times (CO_2 - 420)$, where 420 ppm is the outdoor level. Temperature and humidity move slowly towards a target that depends on the number of people. Every room has its own class schedule and ventilation rate, and random noise is added to every measurement.

2.2 Dirty data and injected faults

Real sensor data is never clean, so the generator also produces typical problems. Some of them are data errors that the cleaning step must handle, and some are real faults that the detection rules must find. Table 2 lists all of them. Because the generator uses a fixed random seed (42), I know exactly what was injected, and this helps to evaluate the system in Section 6.

Table 2. Problems injected into the stream

Problem	Description	Count / time
Missing value	One of the three fields is empty	53 values
Error code	Impossible values such as -127 C, 255 % or 65535 ppm	16 values
Spike inside valid range	Short jump that looks like a normal value	10 values
Duplicate message	The same message is sent again in the next poll	26 messages
Late message	Message arrives 2 to 4 polls after its timestamp	50 messages
Ventilation failure	Lab-305, CO2 keeps growing	09:15 to 10:15
Heater fault	Lecture-101, temperature rises fast	09:55 to 10:00
Frozen sensor	Library-1F, the same values are repeated	10:20 to 10:35
Lost connection	Canteen sends no messages	10:40 to 10:48

3. System Design

The system is a pipeline of small steps (Figure 1). Data is never processed as one big table. Instead, the acquisition step gives the pipeline one micro-batch per poll, which is a Pandas DataFrame with the messages that arrived together (usually six rows). The pipeline keeps a window with the last 120 readings for every sensor in memory. When a batch arrives, it is added to the window of its sensor, the indicators are recalculated, and only the new rows are passed to the detection step and written to disk.

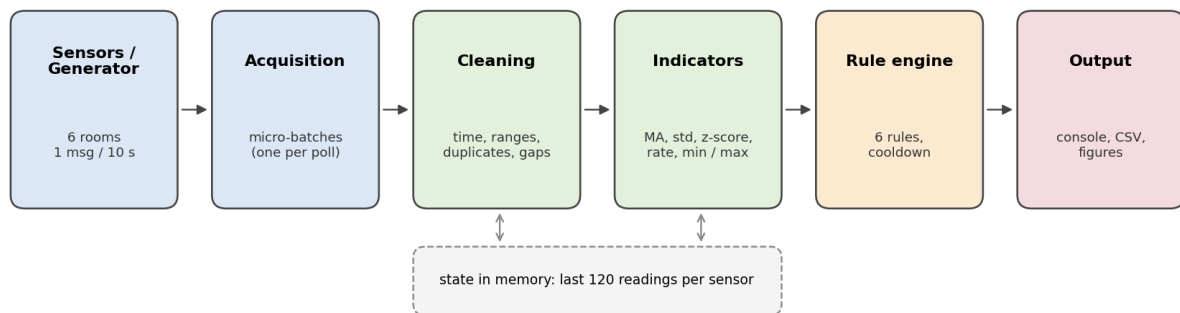


Figure 1. Architecture of the system

There are two data sources, and both give the pipeline the same kind of DataFrame. The first one is the simulator. The second one replays a saved CSV file: it reads the file in chunks of 1000 rows, groups the rows by poll_id and yields one poll at a time, so the whole file is never loaded at once. Because of this, the same processing code works for a live-like stream and for a stored dataset. The code is divided into `config.py`, `generator.py`, `acquisition.py`, `processing.py`, `detection.py`, `pipeline.py`, `storage.py`, `dashboard.py`, `visualize.py` and `evaluate.py`.

4. Implementation

4.1 Cleaning

Every batch goes through the same cleaning steps in `processing.py`:

- The timestamp is converted with `pd.to_datetime`, and rows with a bad timestamp are dropped.
- Values outside the physical range (temperature -10 to 50 C, humidity 0 to 100 %, CO2 300 to 5000 ppm) are replaced by NaN, because they are sensor errors.
- A message with a timestamp that is already in the sensor window is a duplicate and is removed. This also works between different batches.
- Short gaps are filled with the last known value, but at most two readings in a row. Longer gaps stay empty. Each row gets a quality flag: ok, filled or incomplete.

4.2 Real-time indicators

For each sensor I calculate the indicators from Table 3 on the window. All of them are rolling calculations done with Pandas (rolling, shift, ffill) and NumPy (np.maximum, np.where).

Table 3. Indicators

Indicator	Column	Definition
Moving average	*_ma	Mean of the last 30 readings (5 minutes)
Rolling std	*_std	Standard deviation of the last 30 readings
z-score	*_z	Distance of the new value from the mean of the previous 30 readings, in standard deviations
Rate of change	temperature_rate, co2_rate	Change per minute, calculated on a median of 3 readings
Rolling min / max	co2_min_2m, co2_max_5m	Lowest CO2 in the last 2 minutes, highest in the last 5 minutes
Event count / rate	n_records, dashboard	Records per room and minute, and records in the last 60 seconds

The z-score uses the statistics of the previous readings only, so a spike cannot hide itself by increasing the standard deviation. I also set a minimal standard deviation for every metric, otherwise a very calm signal would give huge z-scores for tiny changes:

```
prev = s.shift(1).rolling(WINDOW, min_periods=MIN_PERIODS)
spread = np.maximum(prev.std(), STD_FLOOR[m])
z = (s - prev.mean()) / spread
```

In my first version the rate of change was calculated on the raw values. Then one single spike of +5 C looked like a rise of 4 C per minute and the rule fired without a real problem. Using the median of three readings before the rate fixed this. The z-score is also switched off for the first reading after a gap longer than 60 seconds, because the window is old at that moment.

4.3 Detection rules

There are six rule-based detectors (Table 4). The CO2 rules need the value to stay over the limit for 2 minutes, so a single noisy reading does not create an alert. To avoid hundreds of lines for one long problem, the same alert (sensor, rule, metric) is not repeated during a cooldown of 5 minutes. The offline rule compares the newest message time of each sensor with the current stream time, so it works even when a sensor sends nothing.

Table 4. Detection rules

Rule	Condition	Severity	Alerts
CO2_HIGH	CO2 above 1000 ppm for 2 minutes in a row	warning	22
CO2_CRITICAL	CO2 above 1500 ppm for 2 minutes in a row	critical	2
ZSCORE_SPIKE	abs(z-score) above 6	warning	11
TEMP_RAPID_RISE	Temperature grows faster than 1.2 C per minute	critical	1
SENSOR_STUCK	Temperature, humidity and CO2 identical for 12 readings	warning	3
SENSOR_OFFLINE	No message for more than 60 seconds	critical	1

4.4 Storage and display

Results are written to CSV files while the stream is running, with one append per batch: processed.csv (cleaned values and all indicators), alerts.csv, minute_summary.csv (a one-minute tumbling window per room) and raw_stream.csv (an exact copy of the incoming messages). The console shows every alert as soon as it happens and prints a dashboard with the latest values of all rooms every 60 polls. The figures are created from the stored files by visualize.py.

5. Results

5.1 Console output

Listing 1 shows the dashboard at 10:14:50, when the ventilation failure in Lab-305 was at its worst. The status of the room is critical, and Lecture-101 has a CO2 warning. The counters below the table show how many problems were removed by the cleaning step until this moment.

SMART CAMPUS AIR QUALITY MONITOR				stream time 2025-03-10 10:14:50		poll #450	
Room	Temp C	Hum %	CO2 ppm	CO2 avg 5m	CO2 ppm/min	Status	Last data
Lecture-101	22.6	44.2	1125.0	1131.0	-8.0	WARNING	10:14:50
Lab-201	22.5	41.5	689.0	623.9	35.0	OK	10:14:40
Lab-305	22.8	44.5	1609.0	1562.9	17.0	CRITICAL	10:14:50
Library-1F	21.7	39.1	714.0	706.1	10.0	OK	10:14:50
Canteen	24.9	53.5	962.0	946.9	17.0	OK	10:14:50
Study-Hall	22.4	40.6	864.0	870.8	-5.0	OK	10:14:50

received=2715 processed=2699 duplicates=16 out_of_range=13 missing=38 filled=50							
records in last 60 s: 42 alerts so far: 25							

Listing 1. Console dashboard during the run

5.2 Summary by room

Table 5 gives a summary of the processed data. Peak values are taken from the 5-minute moving average, so that single spikes do not change them.

Table 5. Summary of the processed stream by room

Room	Records	Mean temp, C	Peak temp, C	Mean CO2, ppm	Peak CO2, ppm	Minutes with CO2 > 1000	Alerts
Canteen	672	24.1	25.1	733	986	0	3
Lab-201	720	22.5	22.8	759	864	0	3
Lab-305	720	22.9	23.0	971	1571	51	14
Lecture-101	720	22.7	27.9	942	1137	59	14
Library-1F	720	21.7	21.8	703	734	0	4
Study-Hall	720	22.1	22.5	704	874	0	3

Figure 2 shows the CO2 moving average of all rooms. Lecture-101 and Lab-305 cross the 1000 ppm line, the other rooms stay below it. Lab-305 is the only room that goes over 1500 ppm. The straight line in Canteen between 10:40 and 10:48 is the lost connection. The frozen sensor in Library-1F cannot be seen here, because its moving average is flat anyway, but the SENSOR_STUCK rule found it.

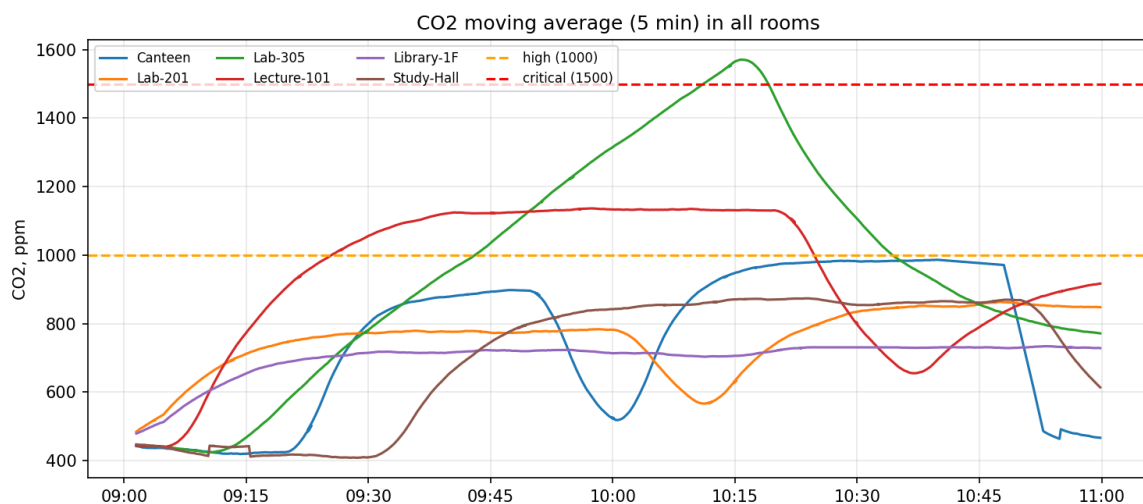


Figure 2. CO2 moving average (5 minutes) in all rooms

Figure 3 is a closer look at Lab-305. The CO2 level grew without stopping from 09:15, when the ventilation failed. The first CO2_HIGH alert came at 09:42, when the level had been over 1000 ppm for two minutes, and

the first CO2_CRITICAL alert came at 10:10. The level went down again after the ventilation was repaired at 10:15.

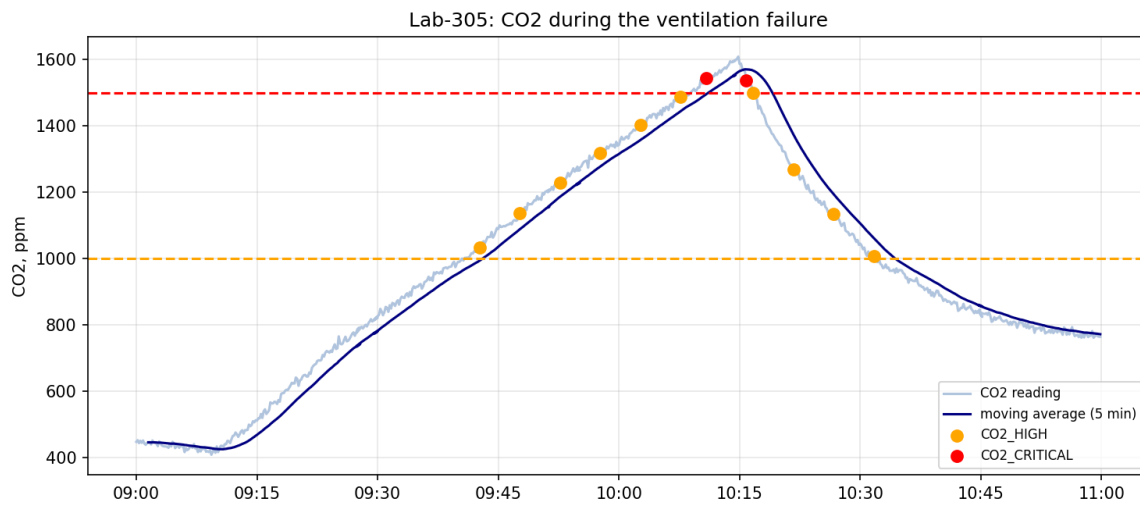


Figure 3. Lab-305: CO2 readings, moving average and alerts

Figure 4 shows the heater fault in Lecture-101. The temperature rose from 22.7 to about 30 C in five minutes. The rate of change went over the limit of 1.2 C per minute at 09:56, and the alert was created 70 seconds after the fault started. The moving average follows the peak with a delay, which is the reason why the rate is a better signal for a fast event.

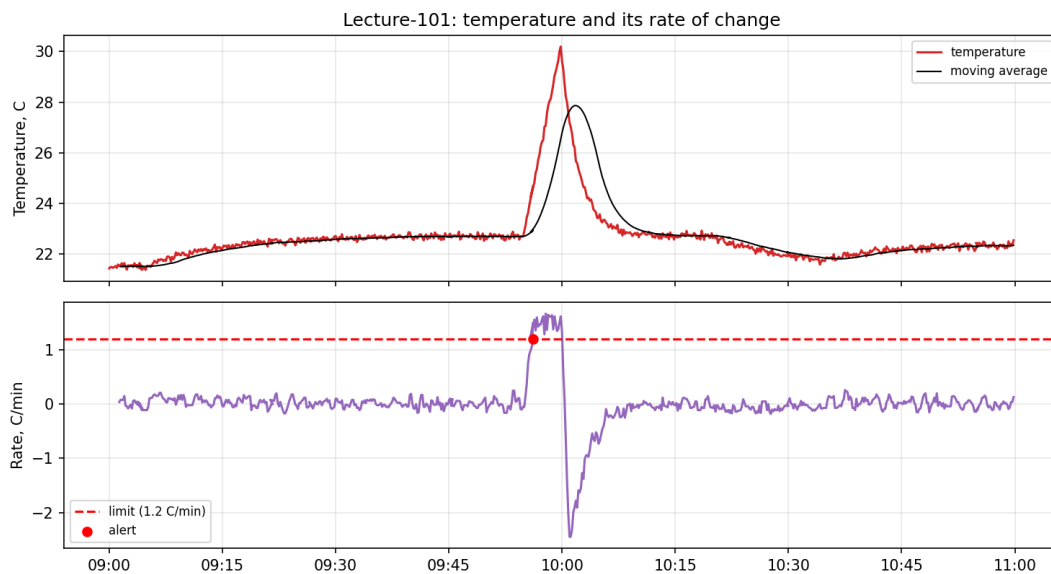


Figure 4. Lecture-101: temperature and its rate of change

6. Evaluation

6.1 Cleaning

The stream contained 4298 raw messages. After cleaning, 4272 rows were left, which is exactly the number of different messages sent by the sensors (6 sensors x 720 polls minus 48 messages lost during the offline period). All 26 duplicates were removed and all 16 error codes were replaced by NaN. The 53 missing values and 16 removed error codes were then filled from the previous reading (69 values in total), so no row stayed incomplete. The late messages were put in the right place by sorting the window on time.

6.2 Detection of faults

Table 6 compares each injected fault with the first alert of the matching rule. For the ventilation failure I measured the time from the first reading above 1500 ppm, because CO2 needs almost an hour to grow. All

four faults were found, with a delay between 60 and 110 seconds. This delay is mostly the persistence that the rules ask for (2 minutes for CO₂, 12 readings for a frozen sensor, 60 seconds for the offline check).

Table 6. Detection of the injected faults

Fault	Rule	Event time	First alert	Delay, s
Ventilation failure (Lab-305)	CO ₂ _CRITICAL	10:09:00	10:10:50	110
Heater fault (Lecture-101)	TEMP_RAPID_RISE	09:55:00	09:56:10	70
Frozen sensor (Library-1F)	SENSOR_STUCK	10:20:00	10:21:40	100
Lost connection (Canteen)	SENSOR_OFFLINE	10:40:00	10:41:00	60

6.3 The z-score threshold

I tested several thresholds for the z-score rule (Table 7). All 10 injected spikes have a very high z-score, so they are found with every threshold. The difference is in the other rows that are flagged. With the usual limit of 3 there are 72 extra rows. Some of them belong to real changes, like the start of a lesson or the heater fault, and the rest is normal noise. With a limit of 6 only one extra row is left (Lecture-101 at 09:55:20, during the heater fault, so it is a real anomaly). I therefore used 6. This choice was made on simulated data with only 10 spikes, so with real sensors the threshold would have to be tuned again.

Table 7. z-score threshold and number of flagged rows

z limit	Flagged rows	Injected spikes found	Recall	Other rows
3	82	10 of 10	1.00	72
4	28	10 of 10	1.00	18
5	16	10 of 10	1.00	6
6	11	10 of 10	1.00	1
8	10	10 of 10	1.00	0
10	10	10 of 10	1.00	0

6.4 Speed and reproducibility

On the machine that I used, the whole stream of 4298 messages was processed in about 62 seconds, which is around 69 messages per second. One poll (six sensors) took 86 ms on average and 159 ms at most. The sensors send a poll every 10 seconds, so the pipeline uses less than 2 % of the available time and could handle many more sensors. To check reproducibility I ran the simulation, then replayed the saved raw file with `--mode replay`. The files `processed.csv`, `alerts.csv` and `minute_summary.csv` were byte for byte identical in both runs. I also wrote 10 unit tests with `pytest` for cleaning, duplicates, gap filling, z-score, stuck sensor, alerts with cooldown, offline detection and the generator. All of them pass.

7. Limitations and Future Work

- The data is simulated. The thresholds and the z-score limit were chosen on this data and are not proven for real sensors.
- The cooldown repeats a long problem every 5 minutes. Lecture-101 got 12 CO₂_HIGH alerts for one hour of high CO₂. A better solution would be one alert when the problem starts and one when it ends.
- Windows are counted in readings, not in time, so many missing readings make a window longer than 5 minutes.
- The state is kept only in memory. After a restart the windows are empty. A message that comes after its minute was closed is not added to `minute_summary.csv`.
- The system could be connected to a real source such as MQTT or a REST API, and a live chart could be added to the dashboard.

8. Conclusion

The project shows a complete streaming workflow in Python: data arrives in small batches, it is cleaned, indicators are calculated on a moving window, six detectors look for problems, and the results are shown in the console and saved to files. All four injected faults were found within two minutes, and the run can be repeated exactly from the saved data. The most useful lessons for me were how much of the work is data cleaning and how a small design mistake, such as calculating the rate on raw values, can create many false alerts.

References

The pandas development team. pandas documentation. <https://pandas.pydata.org/docs/>

NumPy Developers. NumPy documentation. <https://numpy.org/doc/>

The Matplotlib development team. Matplotlib documentation. <https://matplotlib.org/stable/>